

SEG2105 – Fall 2026

Assignment: Object-Oriented Programming

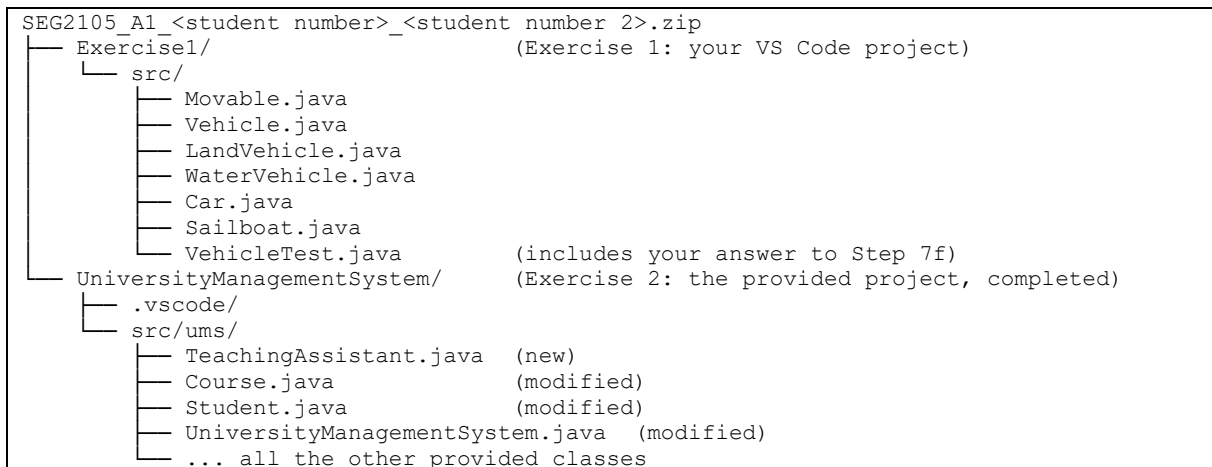
Due date: October 9th, 2026. Before 11h59pm.

Work individually or in Groups of 2.

What to Submit

Submit **one ZIP file** named `SEG2105_A1_<student number1>_<student number2>.zip` (e.g., `SEG2105_A1_300123456_4325668.zip`) on Brightspace **before October 9th, 2026, 11h59pm**.

The ZIP file must contain exactly the two folders below:



Before submitting, make sure that:

- Both projects **compile and run with Java 27 in VS Code**. Code that does not compile will receive a grade of zero for that exercise.
- The output of your Exercise 1 program **matches Appendix 1**.
- The Exercise 2 commands behave as shown in **Appendix 3**.
- Every class, constructor, method, and field has a **Javadoc comment**, and your **name and student number** appear in a comment at the top of every file you created or modified.
- The ZIP file does **not** contain compiled files (`bin` folder, `.class` files) or generated Javadoc (`doc` folder).

Only the last submission made before the deadline will be graded.

Development Environment

All the code for this assignment must be written and run with **Visual Studio Code** and **Java 27** (the latest Java release). Before you start:

1. Install the **Java 27 JDK** (e.g., Eclipse Temurin 27 from adoptium.net). In a terminal, `java -version` must report version 27.
2. Install **Visual Studio Code** (code.visualstudio.com) and the **Extension Pack for Java** (extension id `vscjava.vscode-java-pack`) from the Extensions view.
3. In VS Code, run the command **Java: Configure Java Runtime** (Ctrl+Shift+P) and make sure the project uses JDK 27.
4. For Exercise 1, create a project with **Java: Create Java Project** › **No build tools**, and put all your classes in its `src` folder. Run a program with the **Run** link that appears above its `main` method, or with **Run** › **Run Without Debugging** (Ctrl+F5).

Exercise 1 – Basic Object-Oriented Principles

Objective

You will develop a Java program that focuses on the following key principles of object-oriented programming:

- **Polymorphism:** Implement different behaviours of a single operation (`move`) across a hierarchy of classes, and process objects of different classes through a single `List<Vehicle>`.
- **Constructors:** Implement default and parameterized constructors for each class and demonstrate constructor chaining.
- **Class Variables:** Use class-level (static) variables to track the number of instances created for specific classes.
- **Interfaces:** Implement an interface that enforces a common behaviour across multiple classes.
- **Abstract Classes:** Use an abstract class to enforce method overriding in subclasses, and call an abstract method from a concrete method of the abstract class.

Instructions

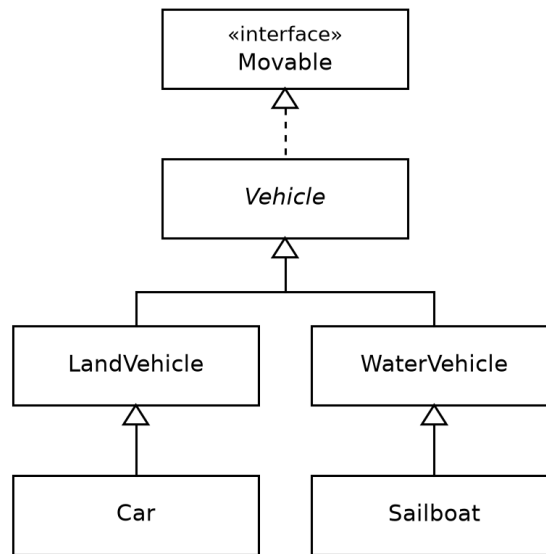


Figure 1 – Class Hierarchy for Vehicle

For this exercise, you will create the class hierarchy shown in Figure 1, which consists of the `Movable` interface, the abstract class `Vehicle`, and the classes `LandVehicle`, `WaterVehicle`, `Car`, and `Sailboat`. Follow the step-by-step instructions provided to implement the hierarchy.

Step 1: Define the Interface `Movable`

1. Create an interface called `Movable`.
2. In the `Movable` interface, define the following method signature:

```
void move();
```

This interface will be used to enforce that all vehicles must have a `move()` method.

Step 2: Define the Abstract Base Class `Vehicle`

1. **Create an abstract class** called `Vehicle` that implements the `Movable` interface.
2. **Fields:**
 - a. Create a **private static** (class-level) variable `numberOfVehicles` to track the total number of vehicles created.

```
private static int numberOfVehicles = 0;
```
 - b. Create a **private** non-static field `name` of type `String` to store the name of the vehicle.
3. **Constructors:**
 - a. Implement a **default constructor** that:
 - o Increments `numberOfVehicles` by 1.
 - o Prints `"Vehicle Constructor called"`.
 - o Initializes the `name` field to `"Unnamed Vehicle"`.
 - b. Implement a **parameterized constructor** that:

- o Accepts a `String` parameter for the vehicle's name.
- o Increments `numberOfVehicles` by 1.
- o Prints "Vehicle Constructor with name called".
- o Initializes the `name` field with the provided value.

4. **Methods:**

- a. Define an **abstract method** `getVehicleType()` which will return a `String` representing the type of vehicle.
- b. Implement a method `getName()` that returns the name of the vehicle. Since `name` is private, subclasses must use this method to access it.
- c. Implement a **concrete method** `describe()` that prints the name of the vehicle followed by its type, by calling `getVehicleType()`:

```
System.out.println(name + " is a " + getVehicleType());
```

Note that `describe()` is written only once, in `Vehicle`, yet its output depends on the actual class of the object.
- d. Implement a **static method** `getNumberOfVehicles()` that returns the total number of vehicles created.

Step 3: Define the Subclass `LandVehicle`

1. **Create a class** called `LandVehicle` that extends `Vehicle`.
2. Create a **private static variable** `numberOfLandVehicles` to track the total number of land vehicles created.
3. **Constructors:**
 - a. Implement a **default constructor** that:
 - o Calls the **default constructor** of `Vehicle` (Hint: lookup the keyword `super`).
 - o Increments `numberOfLandVehicles` by 1.
 - o Prints "LandVehicle Constructor called".
 - b. Implement a **parameterized constructor** that:
 - o Accepts a `String` parameter for the land vehicle's name.
 - o Calls the **parameterized constructor** of `Vehicle`.
 - o Increments `numberOfLandVehicles` by 1.
 - o Prints "LandVehicle Constructor with name called".
4. **Methods:**
 - a. Implement the `move()` method from the `Movable` interface. This method should print:

```
System.out.println("Rolling on land");
```
 - b. Implement the `getVehicleType()` method to return:

```
return "Land Vehicle";
```
 - c. Implement a **static method** `getNumberOfLandVehicles()` that returns the total number of land vehicles created.

Step 4: Define the Subclass *WaterVehicle*

1. **Create a class** called `WaterVehicle` that extends `Vehicle`.
2. **Fields:**
 - a. Create a **private static** variable `numberOfWaterVehicles` to track the total number of water vehicles created.
3. **Constructors:**
 - a. Implement a **default constructor** that:
 - o Calls the **default constructor** of `Vehicle`.
 - o Increments `numberOfWaterVehicles` by 1.
 - o Prints `"WaterVehicle Constructor called"`.
 - b. Implement a **parameterized constructor** that:
 - o Accepts a `String` parameter for the water vehicle's name.
 - o Calls the **parameterized constructor** of `Vehicle` using `super (name)`.
 - o Increments `numberOfWaterVehicles` by 1.
 - o Prints `"WaterVehicle Constructor with name called"`.
4. **Methods:**
 - a. Implement the `move()` method from the `Movable` interface (see Appendix 1 for the expected message).
 - b. Implement the `getVehicleType()` method.
 - c. Implement a **static** method `getNumberOfWaterVehicles()` that returns the total number of water vehicles created.

Step 5: Define the Subclass *Car* (Subclass of *LandVehicle*)

1. **Create a class** called `Car` that extends `LandVehicle`.
2. **Constructors:**
 - a. Implement a **default constructor** that:
 - o Calls the **default constructor** of `LandVehicle`.
 - o Prints `"Car Constructor called"`.
 - b. Implement a **parameterized constructor** that:
 - o Accepts a `String` parameter for the car's name.
 - o Calls the **parameterized constructor** of `LandVehicle`.
 - o Prints `"Car Constructor with name called"`.
3. **Override** the `move()` and `getVehicleType()` methods.

Step 6: Define the Subclass *Sailboat* (Subclass of *WaterVehicle*)

1. **Create a class** called `Sailboat` that extends `WaterVehicle`.
2. **Constructors:**
 - a. Implement a **default constructor** that:
 - o Calls the **default constructor** of `WaterVehicle`.
 - o Prints `"Sailboat Constructor called"`.

- b. Implement a **parameterized constructor** that:
 - o Accepts a `String` parameter for the sailboat's name.
 - o Calls the **parameterized constructor** of `WaterVehicle`.
 - o Prints "Sailboat Constructor with name called".
- 3. **Override** the `move()` and `getVehicleType()` methods.

Step 7: Demonstrate Polymorphism, Constructor Chaining, and Class Variables

In the `main` method of a `VehicleTest` class:

- a. **Create lists** (e.g. array lists) for each **concrete** type of vehicle (`LandVehicle`, `WaterVehicle`, `Car`, `Sailboat`).
- b. **For each list**, create and add **two instances** using default constructors and **two instances** using parameterized constructors (total of **four instances** per class). Use the names shown in Appendix 1.
- c. **Iterate** over each list and call the `move()` method on every object.
- d. Add all the objects created above to a **single** `List<Vehicle>` called `fleet`. Iterate over `fleet` and call `describe()` on every object.
- e. **After all instances are created**, print the total number of created instances for each class by calling their respective static methods (i.e., `getNumberOfVehicles()`, `getNumberOfLandVehicles()`, `getNumberOfWaterVehicles()`).
- f. Unlike the other classes, no list of objects created with `new Vehicle()` is required. In a comment at the top of `VehicleTest`, explain in one or two sentences why such objects cannot be created, and why a `List<Vehicle>` can nevertheless store every object of this program.

Expectations

Make sure to use the appropriate access modifiers (`private`, `protected`, `public`) for variables and methods to promote encapsulation.

Add comments to your code so that JavaDocs can be generated. Include descriptions for classes, constructors, methods, and fields. You can check your comments by running the following command in the VS Code terminal, from the project folder:

```
javadoc -d doc src/*.java
```

Appendix 1 provides the program's expected output.

Exercise 2 – University Management System

Objective

A key skill in software engineering is having the ability to add features to an existing system. Hence, the objective of this exercise is to push you out of your comfort zone by presenting you with a moderately complex system (compared to the systems from your first-year programming courses). In this exercise, you will read and understand aspects of someone else's code. Additionally, you will add simple features to the existing system.

Instructions

Exercise 2.1

For this exercise, you are provided with a basic University Management System. This system allows the user to create students, employees (e.g., administrators and professors), and courses. Additionally, it allows the user to register a student to a course, assign a professor to a course, and assign tasks to an administrator. The system has a Command Line Interface whose commands are provided in the Appendix 2.

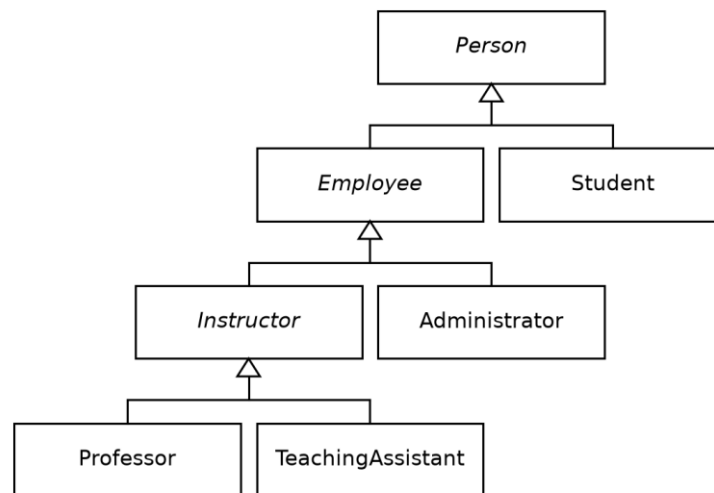


Figure 2 – UML Hierarchy for Person Entities

Step 1: Download and Run the System

Download the VS Code project ([UniversityManagementSystem.zip](#)) that accompanies this assignment, unzip it, and open the `UniversityManagementSystem` folder in VS Code with **File › Open Folder...** (open this folder itself, not its parent). Run the program with **Run › Start Debugging** (F5) using the provided launch configuration *University Management System* (the main method is in the `CommandLineInterface` class). The program runs in the VS Code **Terminal** panel, where you type the commands. Familiarize yourself with the commands described in the Appendix 2. For instance, you may create a course, create a student, create a professor, assign a professor to a course, and register a student to a course.

Note: Do not delete the `.vscode` folder of the project. Its `launch.json` file runs the program in the integrated terminal. The *Debug Console* cannot read the commands you type.

Step 2: Add the TA Functionalities

The entities that represent people (i.e. students, administrators, and professors) are represented in the hierarchy described in the UML diagram of Figure 2. **The `TeachingAssistant` class is not implemented yet. Implement the `TeachingAssistant` class by extending the `Instructor` class.** While a professor can teach 5 courses simultaneously, make sure that a TA can only have a workload of **2 courses** at a time (Hint: study how `Professor` uses the abstract method `getMaxCourses()` of `Instructor`). The `toString()` method of a TA must follow the same format as the one of `Professor`, but start with TA instead of `Professor` (see Appendix 3).

Step 3: Update the Course class

Update the `Course` class so that TAs can be assigned to a course. Unlike professors, a course can have **up to two TAs**. Add a method `addTeachingAssistant(TeachingAssistant ta)` that throws an `IllegalStateException` if the course already has two TAs or if the TA is already assigned to the course. Update the `toString()` method of the course to generate a string that includes a `TAs:` line, placed right after the `Professor:` line, listing the names of the TAs separated by a comma, or `(none)` if the course has no TA (see Appendix 3).

Step 4: Update the `UniversityManagementSystem` Class

1. Study the following methods carefully in the `UniversityManagementSystem` class:
 - o `public void processCreateEmployee(String entity, List<String> input)`
 - o `public void processAssignInstructor(String entity, List<String> input)`
 - o `public void processListEmployees(String entity)`
2. Update the methods listed above to support the following commands:
 - o `create TA "<First name>" "<Last name>" "<salary>"`
 - o `assign TA "<employee id>" "<course code>"`
 - o `list TAs`
 - o `get TA "<employee id>"`

When an assignment is refused (the TA already has 2 courses, or the course already has 2 TAs), **neither** the TA **nor** the course may be modified.

You do not need to add code to the `UniversityManagementSystem` class to support the **get command** as all `Employee` instances are fetched in the same manner regardless of the type of employee.

Note: This update does not require any changes to the `CommandLineInterface` class code as these commands are already supported by the user interface.

Exercise 2.2

Although it is currently possible to register a student in a course, it is not possible to record the student's final grades. Note that the `Student` class already contains a `grades` map that is not used yet.

Step 1: Complete the implementation of the `Student` class

Complete the functionality of the `Student` class so that final grades can be recorded and displayed. Leverage the existing `grades` map. You will need to add at least four methods and update the `toString()` method of the `Student` class:

Method	Description
<code>public void addGrade(String courseCode, int grade)</code>	Records the final grade (0 to 100 inclusively) of the student in a course. Throws an <code>IllegalArgumentException</code> if the grade is out of range or if the student is not registered in the course. Recording a grade for a course that already has one replaces it.
<code>public double getAverage()</code>	Returns the average of the recorded grades, or 0.0 if no grade has been recorded.
<code>public static String toLetterGrade(double grade)</code>	Converts a numeric grade to a letter grade using the table below.
<code>private String getTranscript()</code>	A method that should be invoked by the <code>toString()</code> method to return the student's transcript in the form of a <code>String</code> where each graded course is presented on a separate line (<code>SEG2105: 87 (A)</code>), followed by the average (<code>Average: 82.50 (A-)</code>), or (<code>no grades</code>).
<code>public String toString()</code>	Update the existing method so that the returned <code>String</code> ends with a <code>Transcript:</code> section produced by <code>getTranscript()</code> (see Appendix 3).

Letter grades (University of Ottawa scale):

A+	A	A-	B+	B	C+	C	D+	D	E	F
90–100	85–89	80–84	75–79	70–74	65–69	60–64	55–59	50–54	40–49	0–39

Step 2: Update the `UniversityManagementSystem` Class

Complete the code for the `processAssignGrade` method in the `UniversityManagementSystem` class so that a grade can be recorded. Once the updates are completed, the system should be able to process the following command:

```
assign grade "<student id>" "<course code>" "<grade>"
```

and print a confirmation such as `Grade 87 (A) recorded for Jane Smith in SEG2105.`

Invalid input (unknown student or course, grade that is not a number or out of range, student not registered in the course) must be reported with an error message, as the other commands do.

Note: This update does not require any changes to the `CommandLineInterface` class code as the command is already supported by the user interface.

Appendix 1

Expected output for the Exercise 1 Program:

```
Vehicle Constructor called
LandVehicle Constructor called
Vehicle Constructor called
LandVehicle Constructor called
Vehicle Constructor with name called
LandVehicle Constructor with name called
Vehicle Constructor with name called
LandVehicle Constructor with name called
Vehicle Constructor called
WaterVehicle Constructor called
Vehicle Constructor called
WaterVehicle Constructor called
Vehicle Constructor with name called
WaterVehicle Constructor with name called
Vehicle Constructor with name called
WaterVehicle Constructor with name called
Vehicle Constructor called
LandVehicle Constructor called
Car Constructor called
Vehicle Constructor called
LandVehicle Constructor called
Car Constructor called
Vehicle Constructor with name called
LandVehicle Constructor with name called
Car Constructor with name called
Vehicle Constructor with name called
LandVehicle Constructor with name called
Car Constructor with name called
Vehicle Constructor called
WaterVehicle Constructor called
Sailboat Constructor called
Vehicle Constructor called
WaterVehicle Constructor called
Sailboat Constructor called
Vehicle Constructor with name called
WaterVehicle Constructor with name called
Sailboat Constructor with name called
Vehicle Constructor with name called
WaterVehicle Constructor with name called
Sailboat Constructor with name called

Land vehicles moving:
Rolling on land
Rolling on land
Rolling on land
Rolling on land

Water vehicles moving:
Floating on water
Floating on water
Floating on water
Floating on water

Cars moving:
Vroom! Driving on the road
Vroom! Driving on the road
Vroom! Driving on the road
Vroom! Driving on the road

Sailboats moving:
```

Whoosh! Sailing with the wind
Whoosh! Sailing with the wind
Whoosh! Sailing with the wind
Whoosh! Sailing with the wind

Fleet:

Unnamed Vehicle is a Land Vehicle
Unnamed Vehicle is a Land Vehicle
Tractor is a Land Vehicle
Bus is a Land Vehicle
Unnamed Vehicle is a Water Vehicle
Unnamed Vehicle is a Water Vehicle
Ferry is a Water Vehicle
Canoe is a Water Vehicle
Unnamed Vehicle is a Car
Unnamed Vehicle is a Car
Civic is a Car
Corolla is a Car
Unnamed Vehicle is a Sailboat
Unnamed Vehicle is a Sailboat
Bluenose is a Sailboat
Laser is a Sailboat

Total number of vehicles: 16
Total number of land vehicles: 8
Total number of water vehicles: 8

Appendix 2

Full list of commands supported by the university management system, along with their required parameters and usage. Commands marked with ★ are implemented by you in Exercise 2.

1. Create Commands

These commands allow the creation of various entities within the university management system, including courses, professors, TAs, administrators, and students.

- `create course "<course code>" "<course description>" "<capacity>"`
Creates a new course in the system with a unique course code, description, and specified capacity.
Example usage:
`create course SEG2105 "Introduction to Software Engineering" 165`
- `create professor "<First name>" "<Last name>" "<salary>"`
Creates a new professor entity with the provided first name, last name, and salary.
Example usage:
`create professor "John" "Doe" "85000"`
- `create TA "<First name>" "<Last name>" "<salary>" ★`
Creates a new teaching assistant entity with the provided first name, last name, and salary.
Example usage:
`create TA "Tina" "Tran" "18000"`
- `create administrator "<First name>" "<Last name>" "<salary>"`
Creates a new administrator entity with the given first name, last name, and salary.
- `create student "<First name>" "<Last name>" "<program of study>"`
Creates a new student entity with the given first name, last name, and program of study.

2. Assign Commands

These commands are used to assign professors, TAs and students to courses, administrators to their tasks, and grades to students.

- `assign professor "<employee id>" "<course code>"`
Assigns a professor with the specified employee ID to a course using the course code.
Example usage:
`assign professor "0000001" "SEG2105"`

- `assign TA "<employee id>" "<course code>" ★`
Assigns a TA with the specified employee ID to a course using the course code.
- `assign student "<student id>" "<course code>"`
Registers a student with the provided student ID to a course using the course code.
- `assign administrator "<employee id>" "<task>"`
Assigns a task to the administrator with the provided employee ID.
Example usage:
`assign administrator "0000002" "Book the exam rooms"`
- `assign grade "<student id>" "<course code>" "<grade>" ★`
Records the final grade (0 to 100) of a student in a course in which the student is registered.
Example usage:
`assign grade "300000001" "SEG2105" "87"`

3. List Commands

The list commands display all instances of the specified entity type in the system.

- `list employees`
Displays all employees (including professors, TAs and administrators) currently stored in the system.
- `list administrators`
Displays all administrators in the system.
- `list professors`
Displays all professors in the system.
- `list TAs ★`
Displays all TAs in the system.
- `list students`
Displays all students in the system.
- `list courses`
Displays all courses available in the system.

4. Get Commands

The get commands allow users to retrieve detailed information for a specific entity based on their unique ID or course code.

- `get employee "<employee id>"`
- `get administrator "<employee id>"`
- `get professor "<employee id>"`
- `get TA "<employee id>" ★`

- `get student "<student id>"`
- `get course "<course code>"`

5. **Other Commands**

- `help` Displays the list of commands.
- `exit` Exits the program.

Appendix 3

Example of a session with the completed system (Exercises 2.1 and 2.2). Employee and student ids are assigned sequentially, starting at 0000001 and 300000001 respectively. Lines starting with > are typed by the user.

```
> create course SEG2105 "Introduction to Software Engineering" 165
Course created:
SEG2105 - Introduction to Software Engineering
  Enrolment: 0/165
  Professor: (none)
  TAs: (none)
  Students:
    (none)
> create course CSI2110 "Data Structures and Algorithms" 120
Course created:
CSI2110 - Data Structures and Algorithms
  Enrolment: 0/120
  Professor: (none)
  TAs: (none)
  Students:
    (none)
> create course CSI2132 "Databases I" 120
Course created:
CSI2132 - Databases I
  Enrolment: 0/120
  Professor: (none)
  TAs: (none)
  Students:
    (none)
> create TA "Tina" "Tran" "18000"
Employee created:
TA Tina Tran (Employee ID: 0000001)
  Salary: $18,000.00
  Courses (0/2):
    (none)
> create student "Jane" "Smith" "Software Engineering"
Student created:
Student Jane Smith (Student ID: 300000001)
  Program: Software Engineering
  Registered courses:
    (none)
  Transcript:
    (no grades)
> assign TA "0000001" "SEG2105"
TA Tina Tran assigned to SEG2105.
> assign TA "0000001" "CSI2110"
TA Tina Tran assigned to CSI2110.
> assign TA "0000001" "CSI2132"
Error: Tina Tran already has the maximum of 2 courses.
> get course "SEG2105"
SEG2105 - Introduction to Software Engineering
  Enrolment: 0/165
  Professor: (none)
  TAs: Tina Tran
  Students:
    (none)
> assign student "300000001" "SEG2105"
Student Jane Smith registered in SEG2105.
> assign student "300000001" "CSI2110"
Student Jane Smith registered in CSI2110.
> assign grade "300000001" "SEG2105" "87"
Grade 87 (A) recorded for Jane Smith in SEG2105.
> assign grade "300000001" "CSI2110" "78"
```

```
Grade 78 (B+) recorded for Jane Smith in CSI2110.  
> assign grade "300000001" "CSI2132" "90"  
Error: Jane Smith is not registered in CSI2132.  
> get student "300000001"  
Student Jane Smith (Student ID: 300000001)  
  Program: Software Engineering  
  Registered courses:  
    - SEG2105: Introduction to Software Engineering  
    - CSI2110: Data Structures and Algorithms  
  Transcript:  
    - SEG2105: 87 (A)  
    - CSI2110: 78 (B+)  
  Average: 82.50 (A-)
```